

一方面，操作系统可以根据运行时的内存访问特点，动态设置 TLB 更新与替换策略。但是，这种设计会导致相应的软件设计变得复杂，同时在性能方面也不一定具有优势。

TLB 的引入带来了一个新的挑战：如何保证 TLB 中的缓存项与当前页表中的映射之间的一致性？请思考这个问题：两个应用进程 A 和 B 使用了同样的虚拟地址 VA，但是对应不同的物理地址 PA1 和 PA2。在应用进程 A 访问 VA 时，TLB 会缓存 VA 到 PA1 的翻译；在切换到应用进程 B 运行后，尽管操作系统更新了 CPU 使用的页表基地址，但是当 B 访问 VA 时，CPU 如果依然从 TLB 中寻找 VA 的翻译，则会导致应用进程 B 的 VA 也翻译成 PA1，进而访问了错误的物理内存地址。导致这个问题的根本原因在于：虽然页表已经发生了变化，但 TLB 没有做相应的更新。由于 TLB 是使用虚拟地址进行查询的，所以操作系统在进行页表切换（应用进程切换）的时候需要主动刷掉 TLB，即 TLB 刷新（TLB Flush）^①。这样，在上面的例子中，当 B 访问 VA 时就会发生 TLB 不命中，然后 CPU 会从 B 的页表中取正确的页表项填充 TLB。

若操作系统在切换应用进程的过程中刷新 TLB，那么每当应用进程被切换到的时候，总是会发生 TLB 不命中的情况，进而不可避免地造成性能损失。那么，有没有一种方法使操作系统能够在切换应用进程时不刷掉 TLB 呢？一种为 TLB 缓存项打上“标签”的设计正是为了避免这样的开销。以 AArch64 体系结构为例，它提供了地址空间标识（Address Space ID, ASID）功能^②。具体来说，操作系统可以为不同的应用进程分配不同的 ASID 作为应用进程虚拟地址空间的标签，并将这个标签写入应用进程页表基地址寄存器中的原空闲位（如 TTBR0_EL1 的高 16 位）。TLB 中的缓存项也会包含 ASID 这个标签，从而使得 TLB 中属于不同应用进程的缓存项可以被区分开。MMU 在翻译地址时，会判断 TLB 缓存项中保存的 ASID 与 TTBR0_EL1 中保存的 ASID 是否一致，只有一致时才会使用该 TLB。因此，在切换页表的过程中，操作系统不再需要清空 TLB 缓存项。

小知识：ASID 数量

ASID 最多有 16 位（ASID 位数由 TCR_EL1 寄存器中的配置信息决定），即同时最多可以有 2^{16} （65 536）个标签。若同时运行的应用进程数量超过 ASID 的最大数量，则操作系统不得不为若干应用进程分配相同的 ASID。

不过在修改页表内容之后，操作系统还是需要主动刷新 TLB 以保证 TLB 缓存和页表项内容一致。AArch64 体系结构提供了多种不同粒度的刷新 TLB 的指令，包括刷新全

① “TLB 刷新”是一种常见的翻译，但更准确地说，“TLB Flush”应该是“刷掉 TLB”，也就是将 TLB 设置为无效。

② 在 x86-64 上，类似 ASID 的功能称为进程上下文标识（Process Context ID, PCID）。

部 TLB、刷新指定 ASID 的 TLB、刷新指定虚拟地址的 TLB 等。操作系统可以根据不同场景选用合适的指令，最小化 TLB 刷新的开销，以获得更好的性能。

练习

- 4.8 为什么硬件仅仅采用简单的 TLB 管理方式，就能够在大多数情况下获得较好的 TLB 命中率？
- 4.9 在 AArch64 平台上：
- (a) 若操作系统的页表基地址存在 TTBR1_EL1 中，应用程序的页表基地址存在 TTBR0_EL1 中，那么在发生系统调用时，是否需要 TLB 刷新？
 - (b) 若操作系统为两个应用程序的页表设置了不同 ASID，那么操作系统在切换进程的过程中是否需要 TLB 刷新？
 - (c) 描述操作系统需要进行 TLB 刷新的两个例子。

4.2 操作系统的职责：管理页表映射

本节主要知识点

- ☐ 操作系统需要为自己配置页表吗？
- ☐ 操作系统如何填写应用进程的页表？
- ☐ 操作系统何时填写应用进程的页表？立即映射与延迟映射有什么区别？
- ☐ 缺页异常与段错误（Segmentation Fault）有什么关系？
- ☐ 如何利用虚拟内存实现内存共享和写时拷贝？
- ☐ 使用大页翻译有什么特点？什么时候适合使用大页？

4.2.1 操作系统为自己配置页表

CPU 在上电启动后会默认使用物理地址，这是因为 MMU 的地址翻译功能还未开启，而操作系统则负责在初始化过程中启用该功能。一旦启用 MMU 地址翻译，CPU 会根据页表对指令执行中涉及的地址进行翻译，即认为这些地址都是虚拟地址，因而操作系统和应用进程在后续运行中都是使用虚拟地址。因此，操作系统除了需要为每个应用进程设置页表外，也需要为自己配置页表。

通常，操作系统为自己配置的页表具有两个特点。第一，操作系统一般使用高虚拟地址，比如对于 AArch64 来说，操作系统使用高 16 位为 1 的虚拟地址；应用进程使用低虚拟地址，在 AArch64 中即为高 16 位为 0 的虚拟地址。第二，操作系统一般会一次性将全部物理内存映射到虚拟地址空间中。具体的映射方式非常简单，即虚拟地址 = 物

理地址 + 固定偏移，该映射方式也称为直接映射（Direct Mapping）。操作系统所使用的虚拟地址空间也称为内核地址空间。通过上述固定偏移的页表映射方式，操作系统能够在内核地址空间中很方便地在物理地址和虚拟地址之间进行转换。当操作系统需要访问一个物理地址时，仅需要访问该物理地址加上固定偏移的虚拟地址即可。

以 AArch64 体系结构为例，结合 4.1.3 节介绍的硬件职责，一种常见的配置是：应用进程使用 0x00000000_00000000 ~ 0x0000FFFF_FFFFFFFF 的虚拟地址，这部分地址由页表基地址寄存器 TTBR0_EL1 指向的页表进行翻译；操作系统使用 0xFFFF0000_00000000 ~ 0xFFFFFFFF_FFFFFFFF 的虚拟地址，这部分地址由页表基地址寄存器 TTBR1_EL1 指向的页表进行翻译，如图 4.8 所示。操作系统在初始化过程中，采用固定偏移的地址映射方式为自己配置页表，并且把该页表基地址写入 TTBR1_EL1 中。在之后的运行中，通常 TTBR1_EL1 中的值不需要改变，即始终指向操作系统的页表。操作系统会为每个应用进程创建一张独立的页表，在切换应用进程时，操作系统会把下一个应用进程的页表基地址（物理地址）写入 TTBR0_EL1 中。

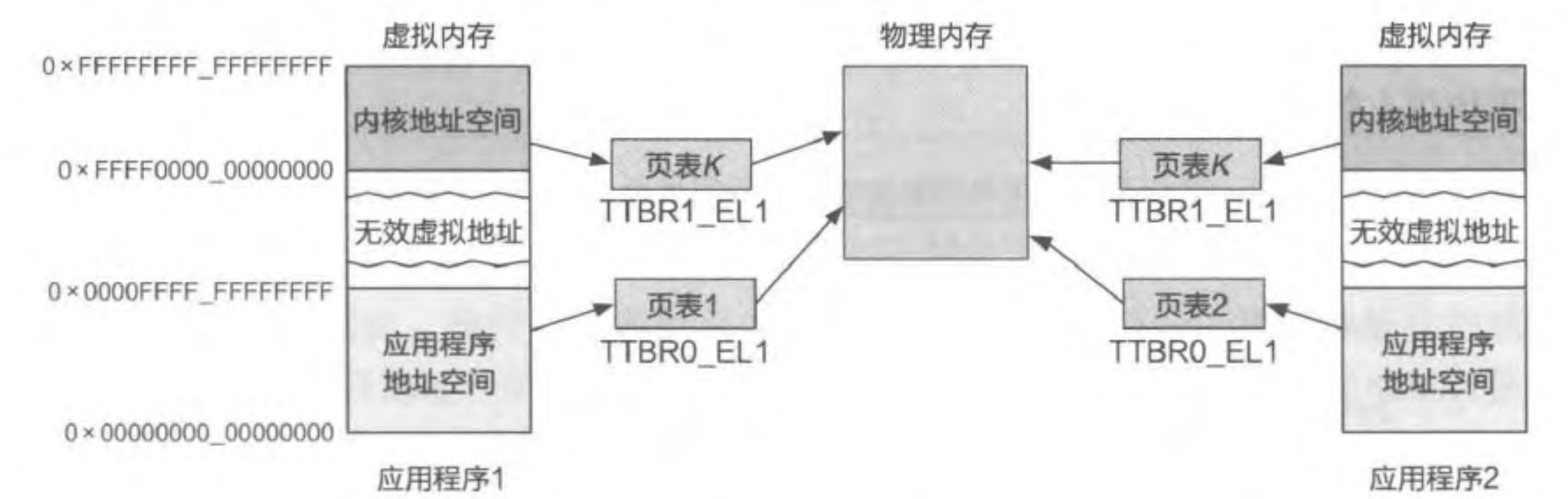


图 4.8 操作系统页表和应用进程页表

练习

4.10 操作系统自己访问内存时，使用虚拟地址还是物理地址？

4.2.2 如何填写进程页表

在进程的执行过程中，所有访存操作用到的虚拟地址均会被 MMU 根据页表翻译为物理地址，而页表的配置则是操作系统的职责。操作系统通过为不同应用进程分配独立页表的方式，为它们提供独立的虚拟地址空间，即不同应用进程可以使用相同的虚拟地址，但经过页表翻译后映射到不同的物理地址，彼此不会干扰。因此，操作系统会把进程页表作为一个不可或缺的成员变量记录在进程结构体中。如代码片段 4.2 所示，进程结构体中包含进程页表基地址成员变量。

代码片段 4.2 进程结构体与添加 / 删除页表映射的函数声明。在进程结构体中，页表基地址是不可或缺的成员变量

```

1 struct process {
2     // 上下文
3     struct context *ctx;
4     // 页表基地址 (物理地址)
5     u64 pgtbl;
6     ...
7 };
8
9 void add_mapping(struct process *, u64 va, u64 pa);
10 void delete_mapping(struct process *, u64 va);

```

操作系统通过设置应用进程的页表来为其配置虚拟地址空间映射。具体来说，操作系统需要实现两个功能：在页表中添加映射和删除映射。代码片段 4.3 给出了操作系统在进程页表中添加虚拟地址到物理地址映射 (add_mapping) 的示意代码。根据虚拟地址依次查找 0 级、1 级、2 级页表页中的页表项，如果相应的页表项为空，说明下一级页表页尚不存在，需要分配新的页表页并填写当前的页表项。最后在 3 级页表页的相应页表项中填上物理地址。

代码片段 4.3 操作系统在页表中添加映射的示意代码 (假设为 4 级页表且不使用大页)

```

1 u64 get_next_pgtbl_page(u64 *pgtbl, u32 index)
2 {
3     u64 pgtbl_entry;
4
5     pgtbl_entry = pgtbl[index];
6
7     if (pgtbl_entry == 0) {
8         // 如果没有相应的页表页 (页表空洞), 则分配页表页
9         pgtbl_entry = alloc_pgtbl_page();
10        pgtbl_page[index] = pgtbl_entry | some_permssion;
11    }
12
13    // 页表项中存储的是物理地址, 而操作系统在运行时使用虚拟地址
14    return paddr_to_vaddr(pgtbl_entry);
15 }
16
17
18 // 在进程页表中添加虚拟地址 va 到物理地址 pa 的映射
19 void add_mapping(struct process *p, u64 va, u64 pa)
20 {
21     u64 *pgtbl_page;
22     u32 index;
23
24
25     // 获取 0 级页表页的起始地址, 即页表基地址
26     // 每个页表页占据 4KB, 包含 512 个页表项

```

```

27  pgtbl_page = (u64 *)paddr_to_vaddr(p->pgtbl);
28
29  // 获取虚拟地址在 0 级页表页中的页表项索引
30  index = L0_INDEX(va);
31  // 获取 1 级页表页的起始地址
32  pgtbl_page = get_next_pgtbl_page(pgtbl_page, index);
33
34  // 获取虚拟地址在 1 级页表页中的页表项索引
35  index = L1_INDEX(va);
36  // 获取 2 级页表页的起始地址
37  pgtbl_page = get_next_pgtbl_page(pgtbl_page, index);
38
39  // 获取虚拟地址在 2 级页表页中的页表项索引
40  index = L2_INDEX(va);
41  // 获取 3 级页表页的起始地址
42  pgtbl_page = get_next_pgtbl_page(pgtbl_page, index);
43
44  // 获取虚拟地址在 3 级页表页中的页表项索引
45  index = L3_INDEX(va);
46  // 在 3 级页表页的页表项中填写物理地址 paddr
47  pgtbl_page[index] = pa | some_permission;
48 }

```

需要注意的是，进程页表基地址和页表项中存储的地址都是物理地址，但操作系统在配置页表的过程中读写页表页所使用的均为虚拟地址，如代码片段 4.3 的第 5 行、第 10 行、第 47 行。因此，操作系统需要具备如下能力：迅速找到一个物理地址在内核地址空间中对应的虚拟地址。示意代码中使用 `paddr_to_vaddr` 接口把物理地址转换成操作系统能够使用的虚拟地址，这个接口的实现正是利用了 4.2.1 节提到的直接映射，即可通过在物理地址上增加一个固定偏移得到虚拟地址，再用虚拟地址访问对应的物理地址。其中，固定偏移是操作系统在配置自己的页表时选择的常量^①。

如代码 4.4 所示，类似于 `add_mapping` 函数，`delete_mapping` 函数根据虚拟地址参数依次查找页表项，最终把第 3 级页表页的相应页表项清空，即可完成地址映射的删除。不同的是，`delete_mapping` 函数在删除映射后，需要刷新 TLB，因为 TLB 中可能缓存了该虚拟地址之前的映射。

代码片段 4.4 操作系统在页表中删除映射的示意代码（假设为 4 级页表且不使用大页）

```

1  u64 find_next_pgtbl_page(u64 *pgtbl, u32 index)
2  {
3      u64 pgtbl_entry;
4
5      pgtbl_entry = pgtbl[index];
6
7      // 没有相应的页表页（页表空洞）

```

① 例如，在 AArch64 上，固定偏移常量可以为 `0xFFFF0000_00000000`。

```
8   if (pgtbl_entry == 0) return 0;
9
10  return paddr_to_vaddr(pgtbl_entry);
11 }
12
13 void delete_mapping(struct process *p, u64 va)
14 {
15     u64 *pgtbl_page;
16     u32 index;
17
18     pgtbl_page = (u64 *)paddr_to_vaddr(p->pgtbl);
19
20     index = L0_INDEX(va);
21     pgtbl_page = find_next_pgtbl_page(pgtbl_page, index);
22     if (pgtbl_page == 0) return;
23
24     index = L1_INDEX(va);
25     pgtbl_page = find_next_pgtbl_page(pgtbl_page, index);
26     if (pgtbl_page == 0) return;
27
28     index = L2_INDEX(va);
29     pgtbl_page = find_next_pgtbl_page(pgtbl_page, index);
30     if (pgtbl_page == 0) return;
31
32     index = L3_INDEX(va);
33     // 参数 va 对应的页表项存在，将该页表项清空
34     pgtbl_page[index] = 0;
35
36     // 利用硬件提供的精准刷新虚拟地址相应 TLB 项的指令
37     flush_tlb(p->pgtbl, va);
38 }
```

在具体实现中，操作系统在为进程创建页表时只需要分配 0 级页表页（即单个物理页），而无须创建完整的页表。在进程运行的过程中，当操作系统需要在进程页表中添加新的映射时，才会根据需要分配新的内存页作为页表页（代码片段 4.3 的第 7 ~ 10 行）。页表页的按需分配体现了多级页表机制能够节约内存的优势。操作系统为 0 级页表页分配内存与为其他级页表页分配内存没有区别，都使用 `alloc_pgtbl_page` 接口完成。由于每个页表页对应一个物理页，因此 `alloc_pgtbl_page` 接口只需要调用 `alloc_page` 分配一个空闲的物理页即可。第 5 章会详细介绍 `alloc_page` 在现代操作系统中的一种主流实现。

练习

- 4.11 `add_mapping` 函数在逐级查找页表项的过程中可能遇到页表页不存在的情况，`delete_mapping` 函数会遇到该情况吗？如果遇到了该如何处理？

4.2.3 何时填写进程页表：立即映射

图 4.9 展示了典型的进程虚拟地址空间布局。如图所示，进程虚拟地址空间包括若干虚拟内存区域，如代码段、数据段、用户堆 / 栈、其他代码库。在应用进程的生命周期中，其虚拟内存空间的变化主要包括：

- 进程创建时。操作系统将应用的二进制文件和动态代码库加载到物理内存，并在应用进程的页表中添加虚拟地址到物理地址的映射（代码和数据），完成初始的虚拟地址空间布局。
- 进程执行时。应用进程要求改变虚拟地址空间，又可以进一步分为三种常见的情况：进程的栈和堆的空间增加或减少；进程加载或卸载其他代码库；进程通过调用 `mmap` 接口增加新的虚拟内存区域，或通过 `munmap` 接口删除已有的虚拟内存区域。
- 进程退出时。操作系统删除应用进程的页表，即删除整个进程虚拟地址空间。

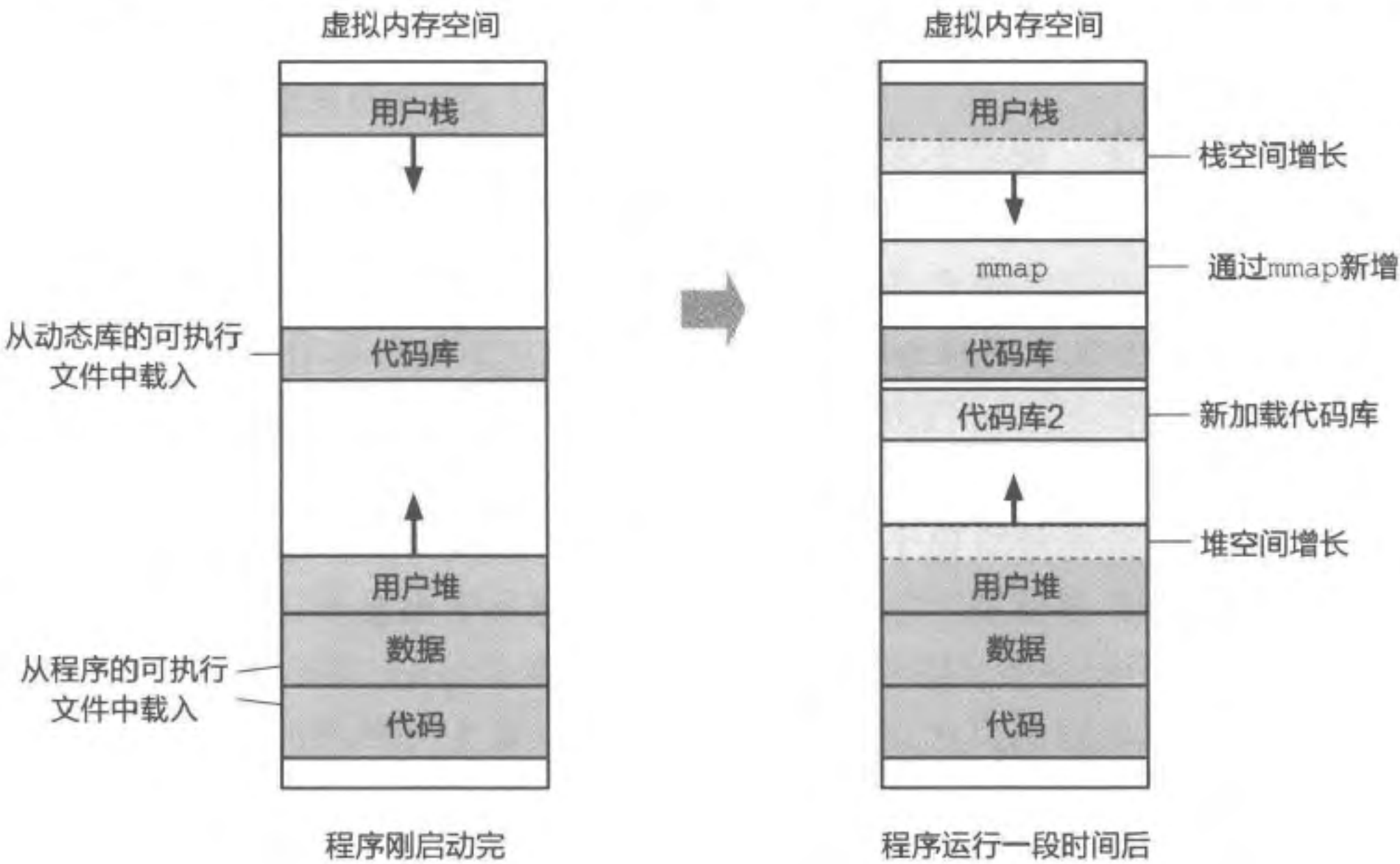


图 4.9 进程虚拟地址空间布局的变化

每个虚拟内存区域由连续的虚拟页组成，操作系统何时在进程页表中为虚拟内存区域添加到物理内存的映射呢？操作系统的一种映射策略是立即映射。在创建应用进程时，操作系统在进程页表中为初始虚拟内存区域（如代码段和数据段）直接添加虚拟页到物理页的映射。具体来说，操作系统可首先通过 `alloc_page` 分配空闲的物理页，然后把磁盘中的应用所需的代码和数据加载到这些物理页中，最后通过 `add_mapping` 在进程页表中添加映射。在应用进程执行时，它初始拥有的虚拟内存区域可能不够用，因此需要创建新的虚拟内存区域。在立即映射策略下，操作系统立即分配物理页，并通过更新页表项，将新增虚拟内存区域中每个新的虚拟页映射到物理页。

mmap 接口常被应用进程用于创建新的虚拟内存区域。代码片段 4.5 的第 5 行给出了 mmap 函数接口，第 11 行是调用 mmap 函数的示例。应用进程通过 mmap 接口告诉操作系统分配一块虚拟内存区域。mmap 接收的第一个参数是一个虚拟地址，应用进程可以通过该参数指明新分配虚拟内存区域的起始地址；第二个参数表示新分配虚拟内存区域的大小；第三个参数表示新分配虚拟内存区域的访问权限；其余参数在此不再赘述。操作系统在接收到应用进程 mmap 的请求后，直接在进程页表中为应用进程添加虚拟页到物理页的映射。一种实现如代码片段 4.6 所示[⊖]，操作系统为应用进程申请的虚拟内存区域中的每个虚拟页依次分配物理页，并且在页表中添加映射。

代码片段 4.5 简单的应用进程 hello-mmap

```

1 #include <stdio.h>
2 #include <string.h>
3 #include <sys/mman.h>
4
5 // void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
6
7 int main(void)
8 {
9     char *buf;
10
11     buf = mmap((void *)0x500000000, 0x2000, PROT_READ | PROT_WRITE, MAP_
        ↪ ANONYMOUS | MAP_PRIVATE, -1, 0);
12     printf("mmap returns %p\n", buf);
13
14     strcpy(buf, "Hello, mmap");
15     printf("%s\n", buf);
16
17     return 0;
18 }
19
20 The output after the execution is like :
21 mmap returns 0x500000000
22 Hello, mmap

```

代码片段 4.6 操作系统中 mmap 的示意代码实现（直接映射）

```

1 // 参数 addr 和 length 分别是虚拟内存区域的起始地址和长度
2 // 示意代码忽略边界条件检查等
3 void sys_mmap(u64 addr, u64 length, ...)
4 {
5     u64 page_num;
6     u64 pa;
7     struct process *proc;
8
9     // 总共需要映射的页面数量

```

⊖ 为保持简洁清晰，本章示意代码均省略了边界条件检查。


```

10  page_num = get_page_num(length);
11
12  // 获取当前进程 (即发起系统调用的进程)
13  proc = get_current_process();
14
15  // 为每个虚拟页分配物理页, 并在页表中添加映射
16  while (page_num > 0) {
17      pa = alloc_page();
18      add_mapping(proc, addr, pa);
19      addr += PAGE_SIZE;
20      --page_num;
21  }
22 }

```

综上所述, 若操作系统采用立即映射的页表填写策略, 则会在初始化一个进程的虚拟地址空间时, 直接在进程页表中添加各虚拟内存区域的映射。在进程运行期间, 操作系统在接收到进程发起的创建虚拟内存区域的请求后, 立刻在进程页表中为该区域中的每个虚拟页添加到物理页的映射。

立即映射策略虽然简单直观, 但在现实中会带来不少问题, 主要是启动时延的增加和内存的浪费。让我们思考两个场景。第一, 一个大型单机游戏程序中包含大量的代码和数据, 不过每次玩家玩游戏的时候实际涉及的代码和数据都是有限的 (比如闯过某关卡后才会继续), 若操作系统采用立即映射的策略, 那么在启动游戏 (创建游戏进程) 时就需要为所有代码和数据分配物理内存并进行加载。这种做法的问题在于操作系统为游戏进程分配的物理内存大多数在实际游戏运行过程中并不会被用到, 既浪费了物理内存资源, 又造成游戏启动时间长。第二, 小明编写的一个应用进程在运行时调用 `mmap` 接口分配虚拟内存区域, 不过由于小明无法预测在运行时实际需要分配多大内存, 所以在编写应用时就把 `mmap` 的 `length` 参数设置为 1GB (应用可能使用的最大内存量)。但是在该应用实际运行的过程中, 上述 `mmap` 所分配的虚拟内存区域的大部分虚拟页都不会被用到。若操作系统采用立即映射的策略, 在应用进程调用 `mmap` 后直接为整个新分配的虚拟内存区域分配物理页、填写页表, 会造成大量物理内存资源的浪费。

那么, 操作系统该如何解决立即映射策略所带来的问题呢? 一种直观的想法是, 如果操作系统可以根据应用进程在运行过程中的实际需要进行物理页分配和页表填写, 那么就可以避免分配的物理页实际不被用到的情况。为此, 操作系统设计了延迟映射的策略, 将虚拟内存的分配与物理内存的分配解耦开。

练习

4.12 进程的虚拟地址空间由若干段虚拟内存区域组成, 是否意味着分段地址翻译机制实际上比分页地址翻译机制更加合适?

4.2.4 何时填写进程页表：延迟映射

与立即映射策略不同，延迟映射策略的主要思路是：先记录为应用进程分配的虚拟内存区域，但不分配相应的物理内存，当然也不会页表中填写映射；当应用进程实际访问某个虚拟页时，由于页表中没有映射，CPU 会触发缺页异常，操作系统在缺页异常的处理函数中为该虚拟页分配物理页，并在页表中添加映射，然后重新运行触发异常的指令。

为了记录应用进程的虚拟内存区域，需要对进程结构体做出修改，不能仅仅记录页表基地址。如代码片段 4.7 所示，进程结构体中包含描述进程虚拟地址空间的成员变量 `vmpace`。`vmpace` 结构体中除进程页表基地址外，还有一个描述进程虚拟地址空间中各虚拟内存区域的成员变量。每个虚拟内存区域由 `vmregion` 结构体表示，需要包括该区域的起始地址、结束地址、访问权限。

代码片段 4.7 扩展进程结构体中和虚拟内存相关的成员变量

```

1 struct process {
2     // 上下文
3     struct context *ctx;
4
5     // 虚拟内存
6     struct vmpace *vmpace;
7
8     ...
9 };
10
11 struct vmpace {
12     // 页表基地址
13     u64 pgtbl;
14
15     // 若干虚拟内存区域组成的链表
16     list vmregions;
17 };
18
19 // 表示一个虚拟内存区域
20 struct vmregion {
21     // 起始虚拟地址
22     u64 start;
23     // 结束虚拟地址
24     u64 end;
25     // 访问权限
26     u64 perm;
27 };

```

操作系统通过虚拟内存抽象为每个应用进程提供独立而连续的虚拟地址空间。然而，应用进程在实际运行的过程中，往往只会使用整个地址空间中的部分区域。以本章开头

的 `hello-vm` 程序为例，其在运行期间主要的虚拟内存区域如图 4.10 所示，`main` 函数位于代码区域，`str` 字符串位于数据区域。操作系统通过若干 `vmregion` 结构体记录应用进程需要使用的虚拟地址范围。

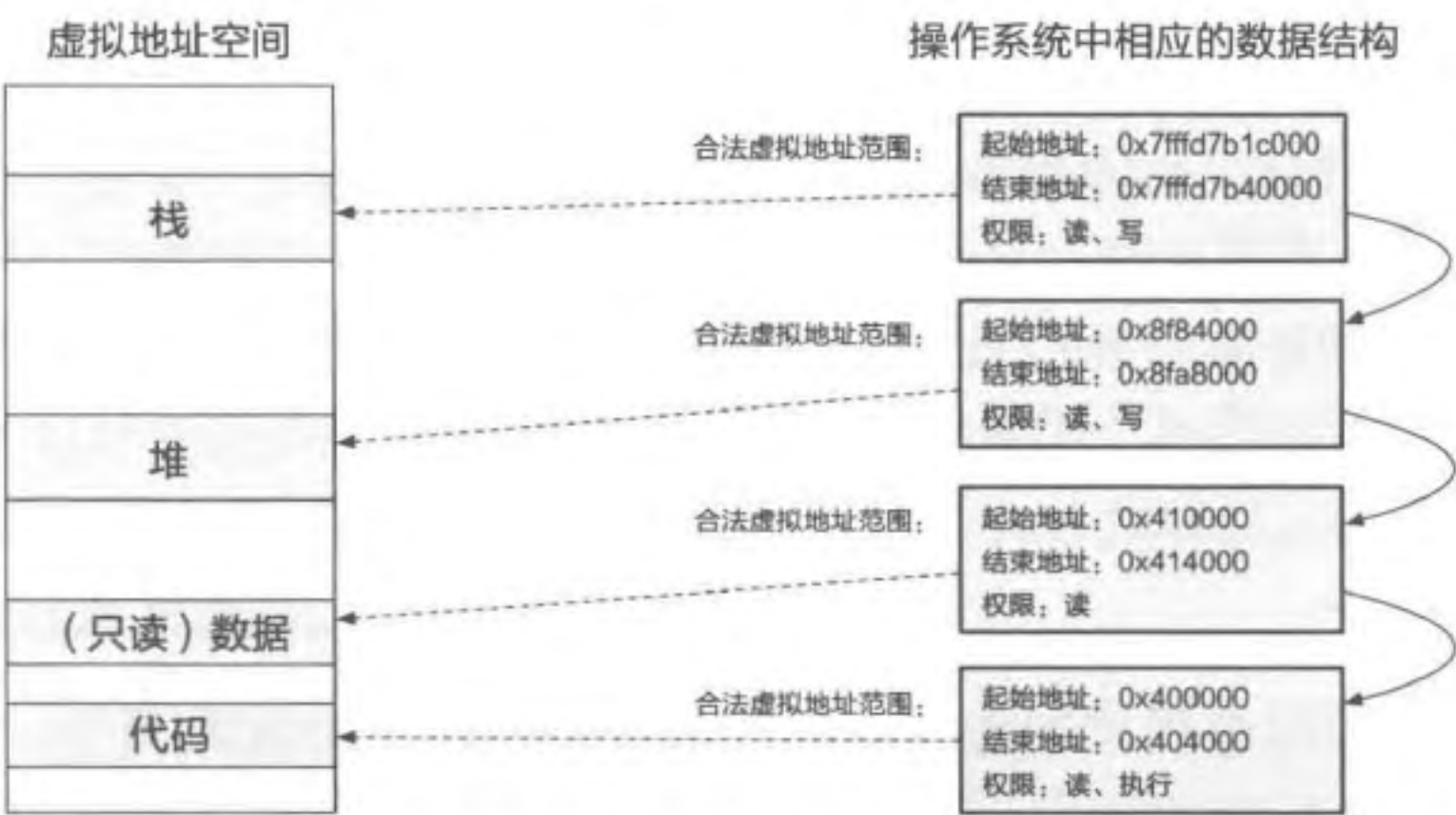


图 4.10 操作系统记录应用进程的虚拟地址空间，图中虚线箭头表示记录关系，实线箭头表示数据结构内部指针

具体来说，操作系统可以分别在创建进程时和进程运行过程中为其添加虚拟内存区域结构体。在第一种情况下，操作系统主动为应用进程设置虚拟内存区域，即操作系统在加载应用进程时为其添加初始的若干虚拟内存区域（例如代码、栈）。

在第二种情况下，应用进程通过操作系统提供的接口添加虚拟内存区域，例如，应用进程在运行期间可以通过调用 `mmap` 接口，显式地要求操作系统添加一段虚拟内存区域。`hello-mmap` 应用进程（代码片段 4.5）的虚拟内存区域在 `mmap` 函数执行前后的变化如图 4.11 所示。采用延迟映射策略后，操作系统中 `mmap` 的实现只需要为进程添加一个 `vmregion` 结构体即可，示意代码如代码片段 4.8 所示。

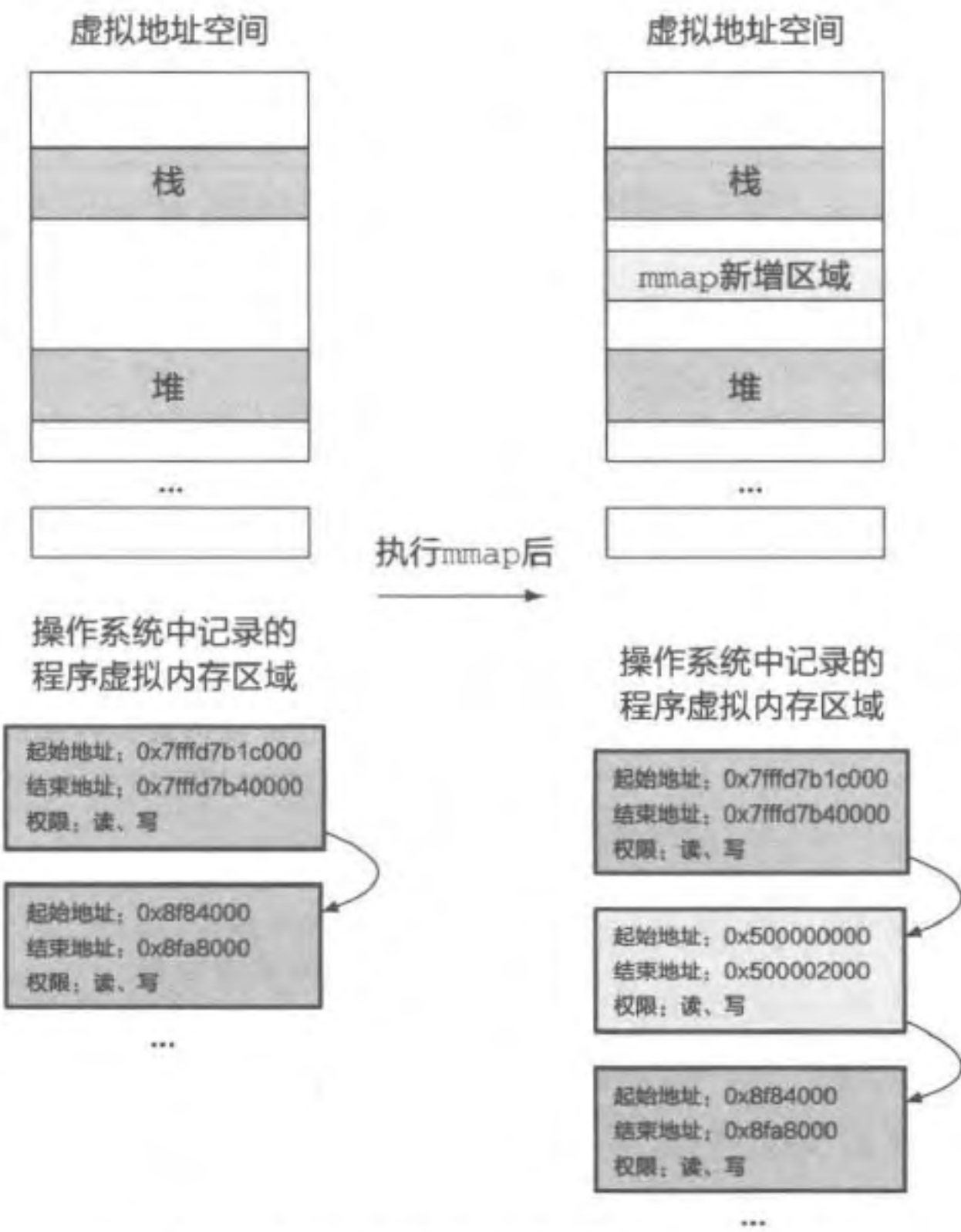


图 4.11 应用进程可以通过调用 `mmap` 接口增加一个虚拟内存区域

代码片段 4.8 操作系统中 mmap 的示意代码实现（延迟映射）

```
1 void sys_mmap(u64 addr, u64 length, u64 perm ...)  
2 {  
3     struct vmregion vmr;  
4     u64 pgtbl;  
5  
6     vmr.start = addr;  
7     vmr.end = addr + length;  
8     vmr.perm = perm;  
9  
10    // 获取当前（发起 mmap 调用）进程的虚拟内存区域链表  
11    vmregions = get_current_process_vmregions();  
12    // 把 vmr 插入 vmregions 链表  
13    add_list(vmregions, vmr);  
14 }
```

操作系统在为应用进程添加虚拟内存区域时并不为其分配物理页、添加页表映射。因此，应用进程在执行期间首次访问某虚拟页会触发缺页异常，操作系统的缺页异常处理函数首先查询触发异常的虚拟地址是否属于某一个虚拟内存区域，并且检查读、写、执行权限是否匹配。在通过这些检查后（合法的缺页异常），则为该虚拟页分配物理页并在页表中添加映射。代码片段 4.9 展示了缺页异常处理函数的示意代码。

代码片段 4.9 操作系统为延迟映射而实现的缺页异常处理函数

```
1 void page_fault_handler(u64 fault_va, ...)  
2 {  
3     u64 va, pa;  
4     list vmregions;  
5     struct vmregion vmr;  
6     struct process *proc;  
7  
8     // 获取当前（触发缺页异常）进程的虚拟内存区域链表  
9     vmregions = get_current_process_vmregions();  
10    // 利用 for_each_vmr 宏遍历链表中的每个虚拟内存区域  
11    for_each_vmr (vmr, vmregions) {  
12        // 检查虚拟地址是否属于已分配的虚拟内存区域  
13        if ((va >= vmr.start) && (va < vmr.end)) {  
14            // 检查访问权限  
15            if (check_perm() == false) return;  
16  
17            // 分配物理页并在页表中添加映射  
18            pa = alloc_page();  
19            proc = get_current_process();  
20            add_mapping(proc, addr, pa);  
21        }  
22    }  
23 }
```


最后，回顾上一节最后描述的两个场景：操作系统采用延迟映射策略后，在启动游戏时和服务应用的 `mmap` 请求时，都无须分配物理页，而是根据应用在运行过程中实际触发的缺页异常进行物理页分配和页表映射，从而能够大大提高物理内存资源的利用率。

真实的操作系统通常采用延迟分配的策略，这种策略也称为按需页面分配（Demand Paging）。具体来说，操作系统在创建应用进程时为其创建若干虚拟内存区域，分别对应代码段、数据段、栈等，并且进程初始页表中仅包含 0 级页表页。应用进程开始运行后，会因为取指令和读写数据而访问未映射的虚拟地址，触发缺页异常。操作系统根据为应用进程维护的虚拟内存区域信息，在确认缺页异常合法之后，分配物理内存页并在应用进程的页表中添加对应的映射。在应用进程运行的过程中，操作系统会根据其发起的请求动态添加虚拟内存区域，此时同样不会直接在页表中增加映射，而是在实际触发缺页异常后才按需进行页表填写。

小知识：Linux 记录虚拟内存区域的结构体——VMA

在 Linux 操作系统中，虚拟内存区域结构体是 `vm_area_struct` (VMA)，其中同样包括起始虚拟地址、结束虚拟地址、访问权限等信息。每个应用进程的虚拟地址空间由若干虚拟内存区域构成，Linux 操作系统通过平衡树数据结构把不同的虚拟内存区域组织起来。

相比于立即映射，延迟映射的优势在于不浪费物理内存，那么它有什么缺点吗？应用进程在访问虚拟页时会触发缺页异常，需要操作系统进行处理，这会增加应用运行性能的开销。为了减少缺页异常对应用性能的影响，一方面，操作系统可以采用预先映射的方式减少缺页异常发生的次数，例如在缺页异常处理函数中为连续多个虚拟页添加页表映射（由于应用进程访存具有空间局部性特点，所以操作系统可以预测相邻的虚拟页很可能会被访问）；另一方面，应用进程可以主动告知操作系统需要提前填写页表映射，例如应用进程在调用 `mmap` 接口时可以设置参数，采用立即映射而不是延迟映射的方式。

小知识：段错误

代码片段 4.10 和输出记录 4.2 展示了在 Linux 操作系统上常遇到的段错误。该应用进程访问了虚拟地址 0（即 `NULL`），而操作系统并没有在它的页表中填上虚拟地址 0 对应的物理地址。因此，当 `printf` 函数访问空指针 `p` 时会触发缺页异常。随后，操作系统设置的缺页异常处理函数检查发现，触发异常的虚拟地址 0 不属于应用进程能够访问的虚拟地址范围（非法虚拟地址访问），所以最终报告段错误。值得一提的是，缺页异常不等同于段错误，前者是 CPU 定义的一种硬件行为，而后者是操作系

统定义的错误，两者的关联是（Linux）操作系统在发现缺页异常是由于访问非法虚拟地址造成的之后，会报告段错误。

代码片段 4.10 简单的应用进程 `fault-vm`

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     char *p = NULL;
6     printf("%s\n", p);
7     return 0;
8 }
```

输出记录 4.2 `fault-vm` 的运行输出结果（运行于 AArch64 平台、Linux 操作系统）

```
[user@osbook ~] $ ./fault-vm
Segmentation fault (core dumped)
```

操作系统实际上为应用进程制定了一条规则：虚拟地址需要分配后才能使用。例如，若应用进程 `hello-vm` 像应用进程 `fault-vm` 一样访问空指针，操作系统会发现触发缺页异常的虚拟地址 0 不属于该应用进程的任何一个虚拟内存区域，那么就说明应用进程访问了非法的虚拟地址，因而操作系统会报告段错误。若应用进程 `hello-vm` 第一次访问某个栈地址（例如 `0x7FFFD7B30000`），同样由于页表中没有填写相应的映射而触发缺页异常，此时操作系统会发现该虚拟地址属于应用进程的一个虚拟内存区域（且访问权限匹配），从而需要为应用进程填上相应的映射。

练习

- 4.13 进程的不同虚拟内存区域需要组织在一起，如果采用树作为数据结构，相对于采用链表有什么优势吗？

4.2.5 常见的改变虚拟内存区域的接口

回顾图 4.9 中展示的虚拟内存区域的变化，应用进程在运行过程中可以主动要求改变自己的虚拟地址空间布局。操作系统提供相关的系统调用供应用进程使用，本节将进行简要介绍。

应用进程可以通过 `mmap` 在自己的虚拟地址空间中新增一段虚拟内存区域。与 `mmap` 接口相对应的是 `munmap` 接口，可用于删除一段虚拟内存区域，或删除已有虚拟内存区域中的一部分。`munmap` 接收两个参数，第一个参数指定待删除的虚拟内存区域的起始

地址，第二个参数指定待删除的虚拟内存区域的长度。在 `munmap` 的实现中，操作系统不仅删除由参数指定的虚拟内存区域，还要在进程页表中删除该区域对应的映射，并且刷新 TLB。由于 `mmap` 和 `munmap` 接口（API）需要修改操作系统记录的进程虚拟内存区域，所以应用进程每次调用它们时都需要向操作系统发起系统调用（ABI）。通常，应用进程加载和删除代码库时，也是通过 `mmap` 和 `munmap` 接口。

虚拟内存区域的变化还包括堆空间的增長和栈空间的增長（堆空间和栈空间都是虚拟内存区域）。应用进程在运行时可能需要动态分配对象，例如 C 语言编写的程序会调用 `malloc` 接口分配内存，`malloc` 返回的内存地址即位于堆中。操作系统在创建应用进程时，可以先为它设置一个大小为 0 的堆，应用进程在运行时，可根据需要，通过系统调用向操作系统申请增大/减小堆的大小。比如，由于堆的初始大小为 0，当应用进程第一次调用 `malloc` 时，会发现堆上没有空闲区域，此时应用进程需要发起系统调用 `brk` 或 `sbrk` 请求操作系统扩展堆的大小。`brk` 接收一个虚拟地址参数，指定新的堆顶位置。`sbrk` 也接收一个参数，用于指定堆大小的变化。它们既可以增大堆大小，也可以减小堆大小。由 `malloc` 分配出来的堆中内存可以被 `free` 接口释放，并且被之后的 `malloc` 接口再次分配。`malloc` 和 `free` 接口不需要操作系统参与，只有需要修改堆大小的 `brk` 或 `sbrk` 接口才要求操作系统参与，这将涉及系统调用。关于堆还有两点值得注意。第一，`malloc` 接口返回的也是虚拟内存（位于堆虚拟内存区域中），操作系统负责为堆中的虚拟页分配物理页并在页表中添加映射。第二，一般来说，应用开发者编写代码时不需要调用 `brk` 或 `sbrk` 接口，因为实现 `malloc` 接口的 C 库负责调用这些接口以管理堆，而且 C 库在扩展堆时通常会申请较大的堆空间，从而减少发起系统调用以扩展堆的次数。

对于栈空间，操作系统通常在创建应用进程时就新建一定大小的栈（Linux 默认为 8MB），当然这也是虚拟地址区域，操作系统可以在应用进程运行时按需为栈分配物理页，从而避免由于分配较大的初始栈而造成物理内存浪费。如果应用进程在运行时实际需要的栈大小超过操作系统分配的初始栈大小，那么操作系统也可以为这种非常见情形提供支持，比如提供系统调用接口以允许应用进程增大栈。

小知识：API 与 ABI

通常来说，`mmap`、`munmap`、`malloc`、`free` 接口是由 C 库向应用进程提供的，这些接口属于 API。C 库在实现这些接口时使用 `mmap`、`munmap`、`brk`、`sbrk` 系统调用，这些接口则属于 ABI，由操作系统提供。`mmap`、`munmap` 作为 API 接口和 ABI 接口同名。

4.2.6 虚拟内存扩展功能

虚拟内存抽象使应用程序能够拥有独立而连续的虚拟地址空间。除此之外，虚拟内

存抽象还带来了许多有用的功能，本节将简单介绍其中的部分内容。

共享内存

共享内存 (Shared Memory) 允许同一个物理页在不同的应用程序间共享，如图 4.12 上半部分所示。例如，应用程序 A 的虚拟页 V1 映射到物理页 P，若应用程序 B 的虚拟页 V2 也映射到物理页 P，则物理页 P 是应用程序 A 和应用程序 B 的共享内存。应用程序 A 读取虚拟页 V1 与应用程序 B 读取虚拟页 V2 将得到相同的内容，并且互相能看到对方修改的内容。共享内存的一个基本用途是支持不同的应用程序之间互相通信、传递数据。此外，基于共享内存的思想，操作系统又从中衍生出写时拷贝 (Copy-on-Write) 等功能。

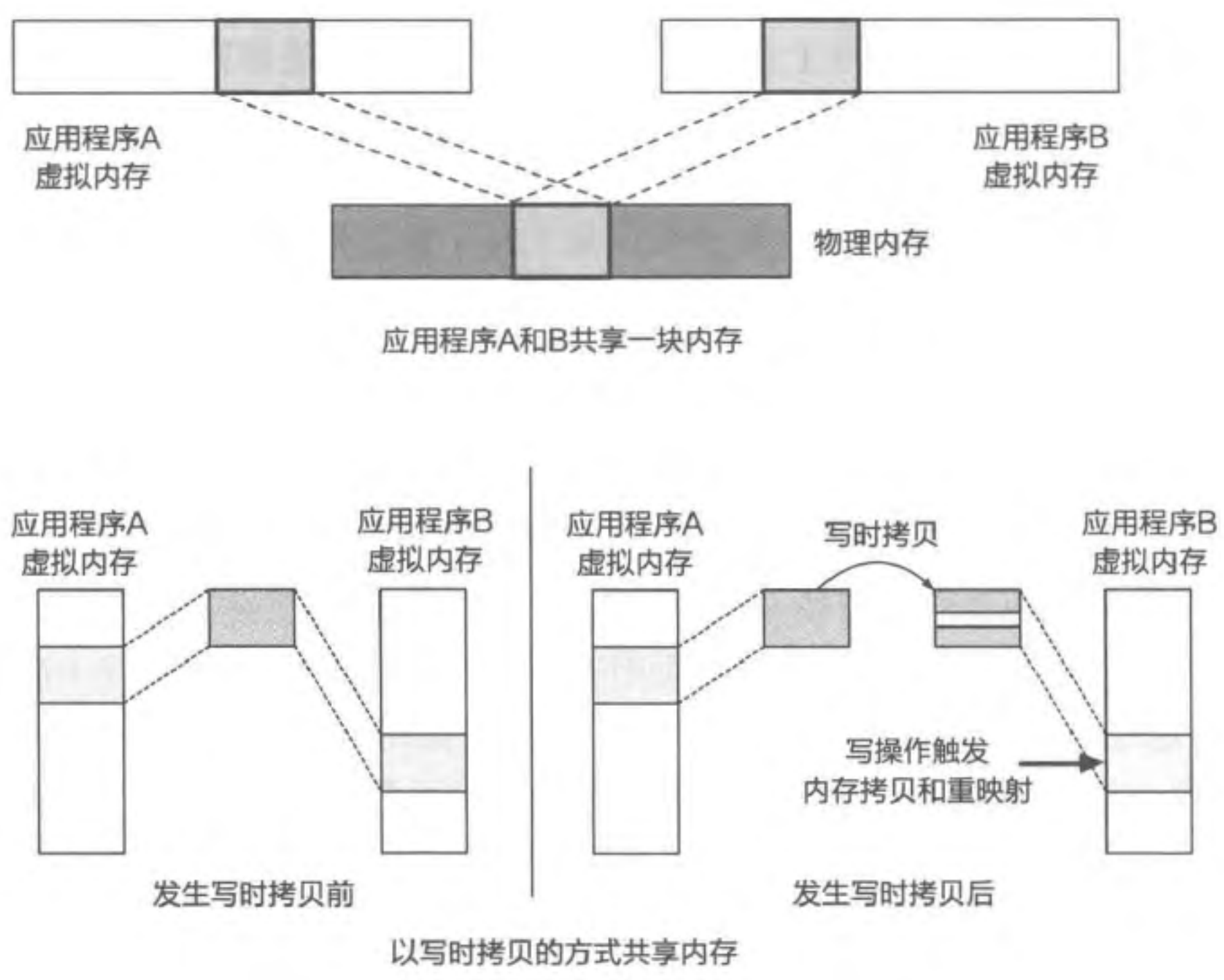


图 4.12 共享内存和写时拷贝

写时拷贝

请思考以下两个场景。第一个场景是两个应用程序拥有很多相同的内存数据（比如加载了相同的动态链接库）。如果把这些数据相同的内存页在物理内存中仅存一份，并以只读的方式映射给两个应用程序，那么就能够显著地节约物理内存资源。第二个场景是在 Linux 操作系统中，一个应用程序可以用 fork 系统调用创建子程序，初始时，父子程序的全部内存数据和地址空间完全一样。如何能够高效地实现这种应用程序创建机制呢？本小节介绍的写时拷贝技术能够很好地应用于这两种场景。

首先再次回顾虚拟内存中的关键数据结构：页表。如 4.1.4 节所述，页表项中除了记录物理页号外，还记录了属性位，包括用于标识虚拟页访问权限的位（比如是否可写、是否可执行）等。写时拷贝正是利用表示“是否可写”的权限位来实现的。

如图 4.12 下半部分所示，写时拷贝技术允许应用程序 A 和应用程序 B 以只读的方式（在页表项中设置只读权限）共享同一段物理内存。一旦某个应用程序对该内存区域进行修改，就会触发访问权限异常。在异常触发后，CPU 同样会将控制流传递给操作系统预先设置的异常处理函数。在该函数中，操作系统会发现当前异常是由于应用程序写了只读内存，而且相应的内存区域又被标记成写时拷贝（例如，Linux 在上文提及的 `vm_area_struct` 结构体中进行标记）。于是，操作系统会将缺页异常对应的物理页重新拷贝一份，并且将新拷贝的物理页以可读可写的方式重新映射给触发异常的应用程序，此后再恢复应用程序的执行。

针对刚刚提到的两个场景，写时拷贝技术一方面能够节约物理内存资源，比如不同的应用程序以写时拷贝的方式映射相同动态链接库的可写部分（动态链接库的只读部分以只读方式在不同应用程序间共享即可）；另一方面，利用写时拷贝可以让父子程序以只读的方式共享全部内存数据，避免内存拷贝操作带来的时间和空间开销。

大页

前文提到，提高 TLB 命中率可以降低地址翻译的性能开销。由于翻译每个内存页都需要占用一个 TLB 缓存项，因此 CPU 中有限的 TLB 缓存项显得弥足珍贵。在内存页大小为 4KB 的情况下，访问 2MB 内存就需要 512 个 TLB 缓存项。随着应用程序对内存容量需求的变大，CPU 中 TLB 缓存项的数量较难保证高 TLB 命中率。

4.1.4 节介绍的大页（Huge Page）机制能够有效缓解 TLB 缓存项不够用的问题。大页的大小可以是 2MB、1GB 甚至 512GB。相比于 4KB 大小的页面，使用大页可以大幅度减少 TLB 的占用量（如访问 2MB 内存只需要 1 个 TLB 缓存项）。

因此，操作系统可利用硬件提供的大页支持，在添加页表映射时以大页进行映射。此外，Linux 还提供了透明大页（Transparent Huge Page）机制，能够自动地将一个应用程序中连续的 4KB 内存页合并成 2MB 的内存页。使用大页的好处主要包括两方面：一方面，它能够减少 TLB 缓存项的使用，从而有机会提高 TLB 命中率；另一方面，它可以减少页表的级数，从而提升查询页表的效率。不过，大页的使用也会增加操作系统管理内存的复杂度，且过度的大页使用还可能存在资源浪费的问题，例如应用程序可能未使用整个大页而造成物理内存资源浪费。

小知识：最小页面大小

除了大页机制外，AArch64 体系结构还支持多种最小页大小，包括 4KB、16KB、64KB。操作系统可以通过 `TCR_EL1` 寄存器进行配置，选择需要的大小。

练习

- 4.14 前文提及，当操作系统选择 4KB 作为最小页大小时，L2 页表项和 L1 页表项对应大页的页面大小分别是 2MB 和 1GB。那么当操作系统选择 16KB 或 64KB 作为最小页大小时，对应大页的页面大小是多少呢？（注意，在 ARMv8.0 上，当使用 16KB 或 64KB 页面大小的时候，只有 L2 页表项支持大页功能。）
- 4.15 操作系统的页表是否适合使用大页进行映射？

4.3 案例分析：ChCore 虚拟内存管理

4.3.1 ChCore 内核页表初始化

由于需要为应用程序提供虚拟内存抽象，操作系统启动期间会启用 CPU 的虚拟内存功能（启用 MMU）。在启用后，MMU 也会对操作系统执行期间使用的地址进行翻译，因此操作系统在启用 MMU 之前需要首先初始化自己的页表。

通常，操作系统对页表的配置分为两个阶段：第一阶段是启动初期，此时操作系统运行在低地址区域，页表将虚拟地址映射为完全相同的物理地址（ $VA=PA$ ）；第二阶段是在跳转到高地址运行后，页表将虚拟地址映射为物理地址加上固定偏移（ $VA=PA+ \text{偏移量}$ ）。下面以本书配套的实验内核 ChCore 在树莓派（AArch64 体系结构）上的页表初始化流程为例，讲解为什么操作系统的页表通常如上进行配置。

如代码片段 4.11 所示，ChCore 在启动后会先把 CPU 特权级别设置为 EL1（对应内核态），然后设置栈帧并调用 C 代码编写的初始化函数。在该函数中，ChCore 首先配置自己的页表（`init_kernel_pt` 函数），其中会配置两份页表（`boot_ttbr0_10` 负责低地址范围的映射，而 `boot_ttbr1_10` 对应高地址范围的映射），然后启用 MMU。代码片段 4.12 展示了如何启用 MMU，`el1_mmu_activate` 函数首先在两个页表基地址寄存器中写入页表基地址，然后将系统寄存器 `sctlr_el1` 的第 0 位设置为 1，即 `SCTLR_EL1_M` 位，从而开启 MMU。注意，从启动第一条指令到 `el1_mmu_activate` 函数中的 `isb` 指令之间，MMU 都未开启，ChCore 使用的是物理地址；而从 `isb` 指令开始，由于 MMU 已经开启，指令地址也会经过地址翻译。假设 `isb` 指令的上一条 `msr` 指令所在的物理地址是 `0x81000`（即存放在物理内存中的位置），那么 `isb` 指令的物理地址则是 `0x81004`。CPU 在执行 `msr` 指令时，PC 中存放的指令地址是 `0x81000`，由于 MMU 尚未启用，CPU 是通过物理地址 `0x81000` 取出该指令的。CPU 执行完 `msr` 指令后，PC 中存放的指令地址是 `0x81004`（准备执行 `isb` 指令），由于 MMU 已经启用，CPU 中的 MMU 会对 PC 中的地址进行地址翻译（把 `0x81004` 作为虚拟地址），而 `isb` 指令的物理地址是 `0x81004`，所以页表中的地址映射需要虚拟地址与物理地址完全相同。